# Spring Boot

## Complete Interview Guide

Basic · Intermediate · Advanced

Detailed Answers · Code Examples · Best Practices · Cheat Sheet

Total Questions: 129 | Sections: 2 | Cheat Sheet Included

**⭐ = Frequently Asked Question**

# Table of Contents

*(Right-click → Update Field in Word to refresh page numbers)*

# Section 1: Spring Boot — Basic Questions (Level I)

*This section covers foundational Spring Boot concepts essential for junior-to-mid level Java developer interviews.*

### ⭐ Q1. What is Spring Boot?

**Answer:**

Spring Boot is an open-source, opinionated framework built on top of the Spring Framework that simplifies the setup, configuration, and deployment of Spring-based Java applications.

- Convention over configuration — sensible defaults eliminate boilerplate XML
- Auto-configuration — automatically configures beans based on classpath dependencies
- Embedded servers — ships Tomcat, Jetty, or Undertow, so no external server is required
- Production-ready features — Actuator endpoints for health checks, metrics, and monitoring
- Standalone JARs — applications run with java -jar without additional deployment steps

---

### Q2. What are the Features of Spring Boot?

**Answer:**

- Auto-Configuration: Configures beans automatically based on classpath and properties
- Starter Dependencies: Curated dependency descriptors (spring-boot-starter-web, etc.)
- Embedded Servers: Tomcat (default), Jetty, or Undertow included in the JAR
- Spring Boot Actuator: HTTP endpoints for health, info, metrics, env, beans
- Spring Initializr: Web-based project scaffolding tool
- Externalized Configuration: application.properties / application.yml with profiles
- DevTools: Hot-reload for faster development cycles
- CLI: Command-line tool to run Groovy scripts

---

### Q3. What are the advantages of using Spring Boot?

**Answer:**

- Rapid Development: Reduces boilerplate and configuration time significantly
- Microservices Ready: Pairs naturally with Spring Cloud for distributed systems
- Large Ecosystem: Integrations with JPA, Security, Kafka, Redis, and more
- Production-Ready: Built-in health checks, metrics, and tracing
- Testability: First-class support for unit and integration tests
- Active Community: Extensive documentation, Stack Overflow support, frequent releases

---

### Q4. Define the Key Components of Spring Boot.

**Answer:**

- Spring Boot Starters: Dependency bundles (e.g., spring-boot-starter-data-jpa)
- Auto-Configuration: @EnableAutoConfiguration + conditional annotations

- Spring Boot CLI: Command-line for rapid prototyping
- Actuator: Monitoring and management endpoints
- Embedded Servers: Tomcat/Jetty/Undertow bundled inside the JAR
- SpringApplication: Entry point class that bootstraps the application

---

### ⭐ Q5. Why do we prefer Spring Boot over Spring?

**Answer:**

Traditional Spring requires extensive XML configuration; Spring Boot eliminates this through auto-configuration and starters.

- No XML config: @SpringBootApplication replaces hundreds of lines of XML
- Embedded server: No need to set up external Tomcat or JBoss
- Starter dependencies: One dependency bundles all transitive dependencies
- Opinionated defaults: Production-ready defaults out of the box
- Faster onboarding: New projects running in minutes via Spring Initializr

---

### ⭐ Q6. Explain the internal working of Spring Boot.

**Answer:**

When you run a Spring Boot application:

- Step 1: SpringApplication.run() is called from the main method
- Step 2: Creates an ApplicationContext (AnnotationConfigServletWebServerApplicationContext for web apps)
- Step 3: Reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports for auto-config classes
- Step 4: Evaluates @Conditional annotations — only relevant beans are registered
- Step 5: Starts the embedded server (Tomcat by default) and binds to the configured port
- Step 6: Publishes ApplicationReadyEvent — application is live

---

### ⭐ Q7. What are the Spring Boot Starter Dependencies?

**Answer:**

Starters are pre-packaged dependency descriptors that bundle all required libraries for a specific feature.

- spring-boot-starter-web: Tomcat + Spring MVC + Jackson
- spring-boot-starter-data-jpa: Hibernate + Spring Data JPA
- spring-boot-starter-security: Spring Security
- spring-boot-starter-test: JUnit 5 + Mockito + MockMvc
- spring-boot-starter-actuator: Monitoring endpoints
- spring-boot-starter-cache: Spring Cache abstraction
- spring-boot-starter-amqp: RabbitMQ integration

---

### Q8. How does a Spring application get started?

**Answer:**

Entry point is a class annotated with @SpringBootApplication containing a main method:

```
@SpringBootApplication
public class MyApp {
    public static void main(String[] args) {
        SpringApplication.run(MyApp.class, args);
    }
}
```

SpringApplication creates the ApplicationContext, triggers auto-configuration, and starts the embedded server.

---

### ⭐ Q9. What does the @SpringBootApplication annotation do internally?

**Answer:**

It is a meta-annotation that combines three annotations:

- @SpringBootConfiguration: Marks the class as a configuration class (like @Configuration)
- @EnableAutoConfiguration: Triggers Spring Boot's auto-configuration mechanism
- @ComponentScan: Scans the current package and sub-packages for Spring components

---

### Q10. What is Spring Initializr?

**Answer:**

Spring Initializr (start.spring.io) is a web-based tool that generates a Spring Boot project skeleton with selected dependencies, build tool (Maven/Gradle), Java version, and packaging type (JAR/WAR). It reduces project setup to seconds.

---

### ⭐ Q11. What is a Spring Bean?

**Answer:**

A Spring Bean is an object managed by the Spring IoC (Inversion of Control) container. Beans are created, configured, and destroyed by the container based on metadata provided through annotations or XML.

- Defined with: @Component, @Service, @Repository, @Controller, or @Bean
- Default scope: Singleton (one instance per ApplicationContext)
- Other scopes: Prototype, Request, Session, Application

---

### ⭐ Q12. What is Auto-wiring?

**Answer:**

Auto-wiring is Spring's ability to automatically inject dependencies into beans without explicit wiring in XML or code. It is achieved via @Autowired.

- Field Injection: @Autowired on a field (not recommended for testability)
- Constructor Injection: Recommended — ensures immutability and easier testing
- Setter Injection: Optional dependencies that can be set after construction

```
// Constructor injection (preferred):
```

```
@Service
public class OrderService {
    private final PaymentService paymentService;
    public OrderService(PaymentService paymentService) {
        this.paymentService = paymentService;
    }
}
```

### Q13. What is ApplicationRunner in Spring Boot?

**Answer:**

ApplicationRunner is a functional interface with a run(ApplicationArguments args) method that executes after the Spring ApplicationContext is loaded. Used for startup logic such as data initialization or health checks.

```
@Component
public class StartupRunner implements ApplicationRunner {
    @Override
    public void run(ApplicationArguments args) {
        System.out.println('App started!');
    }
}
```

### Q14. What is CommandLineRunner in Spring Boot?

**Answer:**

CommandLineRunner is similar to ApplicationRunner but receives raw String[] args instead of ApplicationArguments. Both are used for initialization tasks after the context starts.

- ApplicationRunner: Parses args as key-value pairs (--key=value)
- CommandLineRunner: Receives raw string arguments

### Q15. What is Spring Boot CLI and the most used CLI commands?

**Answer:**

Spring Boot CLI is a command-line tool for quickly prototyping Spring applications using Groovy scripts.

- spring run app.groovy: Run a Groovy script
- spring init --dependencies=web,jpa myproject: Create new project
- spring test app.groovy: Run tests
- spring jar app.jar *.groovy: Package into JAR

### Q16. What is Spring Boot dependency management?

**Answer:**

Spring Boot provides a BOM (Bill of Materials) via spring-boot-starter-parent or spring-boot-dependencies that manages compatible library versions, eliminating version conflicts.

- Inherit spring-boot-starter-parent in pom.xml to get curated dependency versions

- No need to specify versions for managed dependencies — Spring Boot handles compatibility

### ⭐ Q17. Is it possible to change the port of the embedded Tomcat server in Spring Boot?

**Answer:**

Yes. Set the server.port property in application.properties or application.yml:

```
server.port=8081
```

Or pass it at runtime: java -jar app.jar --server.port=8081

To use a random port: server.port=0

### Q18. What happens if a starter includes conflicting library versions?

**Answer:**

Spring Boot's parent POM manages dependency versions through its BOM. If you add a conflicting version manually, Maven's dependency resolution (nearest-wins) or Gradle's conflict resolution determines which version is used. Best practice: always rely on Spring Boot's managed versions unless absolutely necessary and exclude conflicting transitive dependencies explicitly.

### Q19. What is the default port of Tomcat in Spring Boot?

**Answer:**

The default embedded Tomcat port is 8080. This is configured via server.port in application.properties.

### Q20. Can we disable the default web server in a Spring Boot application?

**Answer:**

Yes. Set spring.main.web-application-type=none in application.properties or programmatically:

```
SpringApplication app = new SpringApplication(MyApp.class);
app.setWebApplicationType(WebApplicationType.NONE);
app.run(args);
```

### ⭐ Q21. How to disable a specific auto-configuration class?

**Answer:**

Use the exclude attribute of @SpringBootApplication or @EnableAutoConfiguration:

```
@SpringBootApplication(exclude = { DataSourceAutoConfiguration.class })
```

Or in application.properties:

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.jdbc.DataSourceAutoConfiguration
```

### Q22. Can we create a non-web application in Spring Boot?

**Answer:**

Yes. Set spring.main.web-application-type=none. Use ApplicationRunner or CommandLineRunner for business logic. Useful for batch jobs, CLI tools, and scheduled tasks.

---

### ⭐ Q23. Describe the flow of HTTPS requests through a Spring Boot application.

**Answer:**

- Step 1: Client sends HTTPS request to the embedded Tomcat server
- Step 2: SSL/TLS termination (configured via server.ssl.* properties)
- Step 3: Request passes through Spring Security filter chain (if configured)
- Step 4: DispatcherServlet receives the request and identifies the matching handler
- Step 5: Interceptors (HandlerInterceptor) run pre-handle logic
- Step 6: Controller method processes the request and returns a response
- Step 7: Response passes through MessageConverters (JSON serialization via Jackson)
- Step 8: Response returned to client

---

### ⭐ Q24. Explain @RestController annotation in Spring Boot.

**Answer:**

@RestController is a convenience annotation combining @Controller and @ResponseBody. It marks the class as a controller where every method returns data directly (serialized to JSON/XML) instead of resolving to a view template.

```
@RestController
@RequestMapping('/api/users')
public class UserController {
    @GetMapping('/{id}')
    public User getUser(@PathVariable Long id) {
        return userService.findById(id);
    }
}
```

---

### ⭐ Q25. What is the difference between @Controller and @RestController?

**Answer:**

- @Controller: Returns a view name (resolves to HTML template via ViewResolver)
- @RestController: Returns data directly as JSON/XML (@ResponseBody implicitly applied)
- Use @Controller for MVC web apps with Thymeleaf/JSP
- Use @RestController for REST APIs returning JSON responses

---

### Q26. What is the difference between @RequestMapping and @GetMapping?

**Answer:**

- @RequestMapping: Generic mapping for all HTTP methods (GET, POST, PUT, DELETE)
- @GetMapping: Shorthand for @RequestMapping(method = RequestMethod.GET)

- Similarly: @PostMapping, @PutMapping, @DeleteMapping, @PatchMapping exist

```
@RequestMapping(value='/users', method=RequestMethod.GET) // verbose
@GetMapping('/users')  // preferred shorthand
```

### Q27. Differences between @SpringBootApplication and @EnableAutoConfiguration?

**Answer:**

- @EnableAutoConfiguration: Only triggers auto-configuration; does not scan components
- @SpringBootApplication: Combines @EnableAutoConfiguration + @ComponentScan + @SpringBootConfiguration
- In practice, always use @SpringBootApplication on the main class

### Q28. How can you programmatically determine active profiles?

**Answer:**

Inject the Environment bean and call getActiveProfiles():

```
@Autowired
private Environment environment;

String[] profiles = environment.getActiveProfiles();
```

Alternatively, inject @Value("${spring.profiles.active:default}") for a single active profile string.

### ⭐ Q29. Mention the differences between WAR and embedded containers.

**Answer:**

- WAR: Deployed to external server (Tomcat, JBoss); requires server management
- Embedded: Server bundled in JAR; deploy with java -jar; simpler DevOps
- WAR: Better for legacy enterprise environments with shared servers
- Embedded: Preferred for microservices and cloud-native applications
- Spring Boot defaults to embedded; WAR packaging requires extending SpringBootServletInitializer

### ⭐ Q30. What is Spring Boot Actuator?

**Answer:**

Spring Boot Actuator adds production-ready management and monitoring capabilities via HTTP or JMX endpoints.

- /actuator/health: Application health status
- /actuator/info: Custom application info
- /actuator/metrics: JVM, HTTP, and custom metrics
- /actuator/env: Environment properties
- /actuator/beans: All Spring beans
- /actuator/loggers: Log level management at runtime
- /actuator/threaddump: Thread dump for debugging

### Q31. How to enable Actuator in Spring Boot?

**Answer:**

Add the dependency:

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

Then expose endpoints in application.properties:

```
management.endpoints.web.exposure.include=health,info,metrics,env
management.endpoint.health.show-details=always
```

### Q32. How to get the list of all the beans in our Spring Boot application?

**Answer:**

Option 1: Via Actuator — GET /actuator/beans

Option 2: Programmatically using ApplicationContext:

```
@Autowired
ApplicationContext context;

String[] beanNames = context.getBeanDefinitionNames();
Arrays.sort(beanNames);
for (String name : beanNames) System.out.println(name);
```

### Q33. Can we check the environment properties?

**Answer:**

Yes, via the Actuator /actuator/env endpoint. Ensure it is exposed:

```
management.endpoints.web.exposure.include=env
```

Programmatically: inject Environment and call getProperty('key').

### Q34. How to enable debugging log in Spring Boot?

**Answer:**

In application.properties:

```
logging.level.root=DEBUG
logging.level.org.springframework=DEBUG
```

Or pass --debug flag: java -jar app.jar --debug

Or in application.properties: debug=true (enables auto-config report)

### Q35. Explain the need of dev-tools dependency.

**Answer:**

spring-boot-devtools enhances the development experience:

- Automatic restart: Detects classpath changes and restarts the context quickly
- LiveReload: Browser auto-refresh via built-in LiveReload server
- Disabled in production: DevTools is automatically disabled when running a packaged JAR
- Property defaults: Disables caching (e.g., Thymeleaf template cache) for development

---

### ⭐ Q36. How do you test a Spring Boot application?

**Answer:**

- Unit Tests: JUnit 5 + Mockito to test individual service/component logic in isolation
- Integration Tests: @SpringBootTest to load the full ApplicationContext
- Web Layer Tests: @WebMvcTest to test controllers with MockMvc
- Data Layer Tests: @DataJpaTest with an in-memory H2 database
- TestRestTemplate / WebTestClient: For end-to-end HTTP tests

```
@SpringBootTest(webEnvironment = WebEnvironment.RANDOM_PORT)
class UserControllerTest {
    @Autowired TestRestTemplate restTemplate;
    @Test
    void getUser() {
        ResponseEntity<User> r = restTemplate.getForEntity('/api/users/1',
User.class);
        assertEquals(HttpStatus.OK, r.getStatusCode());
    }
}
```

---

### Q37. What is the purpose of unit testing in software development?

**Answer:**

- Catches bugs early in the development cycle at low cost
- Acts as living documentation of expected behavior
- Enables safe refactoring with confidence
- Reduces regression bugs in CI/CD pipelines
- Encourages modular, testable design

---

### ⭐ Q38. How do JUnit and Mockito facilitate unit testing in Java projects?

**Answer:**

- JUnit 5: Testing framework providing @Test, @BeforeEach, @AfterEach, assertions, and test lifecycle
- Mockito: Mocking framework to create test doubles for dependencies
- Mock.when().thenReturn() stubs method calls
- Mockito.verify() asserts that specific methods were called

```
@ExtendWith(MockitoExtension.class)
class OrderServiceTest {
    @Mock PaymentService paymentService;
```

```
    @InjectMocks OrderService orderService;
    @Test
    void shouldProcessOrder() {
        when(paymentService.charge(any())).thenReturn(true);
        assertTrue(orderService.process(new Order()));
    }
}
```

### ⭐ Q39. Explain the difference between @Mock and @InjectMocks in Mockito.

**Answer:**

- @Mock: Creates a mock (fake) object of the specified type — no real logic runs
- @InjectMocks: Creates a real instance of the class and injects all @Mock fields into it

Use @Mock for dependencies and @InjectMocks for the class under test.

### ⭐ Q40. What is the role of @SpringBootTest annotation?

**Answer:**

@SpringBootTest loads the complete Spring ApplicationContext for integration testing. Key parameters:

- webEnvironment = MOCK: Default — loads context without a real server
- webEnvironment = RANDOM_PORT: Starts server on random port; use with TestRestTemplate
- webEnvironment = DEFINED_PORT: Starts server on configured port
- classes: Specify configuration classes to limit context loading

### ⭐ Q41. How do you handle exceptions in Spring Boot applications?

**Answer:**

- @ExceptionHandler: Method-level handler in a controller for specific exceptions
- @ControllerAdvice / @RestControllerAdvice: Global exception handler across all controllers
- ResponseEntityExceptionHandler: Base class with default handling for standard Spring MVC exceptions

```
@RestControllerAdvice
public class GlobalExceptionHandler {
    @ExceptionHandler(ResourceNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleNotFound(ResourceNotFoundException ex)
{
        return ResponseEntity.status(404).body(new ErrorResponse(ex.getMessage()));
    }
}
```

### Q42. Explain the purpose of the pom.xml file in a Maven project.

**Answer:**

pom.xml (Project Object Model) is the core configuration file for Maven projects:

- Defines project metadata: groupId, artifactId, version

- Declares dependencies and their versions
- Configures build plugins and goals
- Sets parent POM for dependency management inheritance
- Configures profiles, repositories, and distribution management

### ⭐ Q43. How does auto-configuration play an important role in a Spring Boot application?

**Answer:**

Auto-configuration examines the classpath, existing beans, and properties to conditionally register Spring beans without manual configuration. For example, if spring-boot-starter-data-jpa is on the classpath and a DataSource is configured, Spring Boot auto-creates EntityManagerFactory, TransactionManager, and JPA repositories automatically.

- Uses @Conditional annotations: @ConditionalOnClass, @ConditionalOnMissingBean, @ConditionalOnProperty
- Auto-config classes are listed in META-INF/spring/AutoConfiguration.imports
- Can be overridden by defining your own beans

### Q44. Can we customize a specific auto-configuration in Spring Boot?

**Answer:**

Yes. Define your own bean of the same type — Spring Boot backs off due to @ConditionalOnMissingBean. You can also use properties to tune auto-configuration behavior (e.g., spring.datasource.url to configure DataSource auto-config).

### Q45. How can you disable specific auto-configuration classes in Spring Boot?

**Answer:**

Use exclude in @SpringBootApplication:

```
@SpringBootApplication(exclude = { SecurityAutoConfiguration.class })
```

Or in properties:

```
spring.autoconfigure.exclude=org.springframework.boot.autoconfigure.security.servlet.SecurityAutoConfiguration
```

### Q46. What is the purpose of having a spring-boot-starter-parent?

**Answer:**

- Provides default Maven configurations: Java version, encoding, plugin versions
- Imports the spring-boot-dependencies BOM for version management
- Configures the spring-boot-maven-plugin for JAR packaging
- No need to declare versions for Spring Boot managed dependencies

### Q47. How do starters simplify the Maven or Gradle configuration?

**Answer:**

Instead of declaring dozens of individual dependencies (Spring MVC, Jackson, Tomcat, validation, etc.) and managing their compatible versions, you declare one starter — spring-boot-starter-web — and get all transitive dependencies at compatible versions automatically.

---

### ⭐ Q48. How do you create REST APIs?

**Answer:**

- Annotate class with @RestController
- Map HTTP methods: @GetMapping, @PostMapping, @PutMapping, @DeleteMapping
- Accept request body: @RequestBody
- Path variables: @PathVariable
- Query params: @RequestParam
- Return ResponseEntity for fine-grained status code control

```
@RestController @RequestMapping('/api/products')
public class ProductController {
    @GetMapping public List<Product> all() { return service.findAll(); }
    @PostMapping public ResponseEntity<Product> create(@RequestBody @Valid Product p)
{
        return ResponseEntity.status(201).body(service.save(p));
    }
}
```

---

### ⭐ Q49. What is versioning in REST? What are the ways to implement versioning?

**Answer:**

- URI Versioning: /api/v1/users (most common, simple)
- Query Parameter: /api/users?version=1
- Header Versioning: Custom header — X-API-Version: 1
- Media Type (Content Negotiation): Accept: application/vnd.company.v1+json

---

### ⭐ Q50. What are the REST API Best Practices?

**Answer:**

- Use nouns for resources, not verbs (/users not /getUsers)
- Use correct HTTP methods: GET (read), POST (create), PUT (replace), PATCH (partial update), DELETE
- Return appropriate HTTP status codes: 200, 201, 204, 400, 401, 403, 404, 500
- Version your APIs from the start
- Use pagination for collection endpoints
- Validate input with @Valid and return meaningful error messages
- Implement HATEOAS for discoverability in complex APIs
- Secure endpoints with HTTPS and authentication

---

### Q51. What are the uses of ResponseEntity?

**Answer:**

ResponseEntity wraps the HTTP response, giving full control over status code, headers, and body:

```
return ResponseEntity
    .status(HttpStatus.CREATED)
    .header('Location', '/api/users/' + user.getId())
    .body(user);
```

- Control response status codes explicitly
- Set custom response headers
- Return different types based on conditions

---

### Q52. What should the delete API method status code be?

**Answer:**

- 204 No Content: Deletion successful, no response body (most common)
- 200 OK: If returning the deleted resource in the body
- 404 Not Found: If the resource does not exist

---

### ⭐ Q53. What is Swagger?

**Answer:**

Swagger (now OpenAPI Specification) is a framework for designing, describing, and documenting REST APIs. Spring Boot integrates with Springdoc-OpenAPI to auto-generate interactive API documentation from code.

---

### Q54. How does Swagger help in documenting APIs?

**Answer:**

- Auto-generates OpenAPI spec from @RestController annotations
- Provides Swagger UI — interactive browser-based API explorer
- Supports request/response schema documentation
- Add springdoc-openapi-starter-webmvc-ui dependency
- Access at: http://localhost:8080/swagger-ui.html

---

### Q55. What all servers are provided by Spring Boot and which one is default?

**Answer:**

- Tomcat (default): Included in spring-boot-starter-web
- Jetty: Lightweight alternative
- Undertow: High-performance non-blocking server from JBoss
- Netty: Used with Spring WebFlux for reactive applications

---

### Q56. How does Spring Boot decide which embedded server to use?

**Answer:**

Spring Boot uses @Conditional annotations. It detects which server class is on the classpath. If spring-boot-starter-tomcat is present (included in spring-boot-starter-web by default), Tomcat is selected. To switch to Jetty, exclude Tomcat and add spring-boot-starter-jetty.

---

### Q57. How can we disable the default server and enable a different one?

**Answer:**

```
<!-- Exclude Tomcat, add Jetty -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <exclusions>
        <exclusion>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-starter-tomcat</artifactId>
        </exclusion>
    </exclusions>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-jetty</artifactId>
</dependency>
```

---

### ⭐ QCross. Can we use @Component instead of @Repository and @Service?

**Answer:**

Yes, technically — all three are @Component stereotypes, so Spring will detect and register them. However:

- @Repository: Enables persistence exception translation (converts DB exceptions to DataAccessException)
- @Service: Semantic clarity — marks business logic layer; Spring may add AOP features
- @Component: Generic; use when the class doesn't fit service or repository semantics

Best practice: Use the most specific annotation for clear intent and to benefit from Spring's additional processing.

---

# Section 2: Spring Boot — Intermediate Questions (Level II)

*This section covers advanced topics for senior developer, tech lead, and system design interviews.*

### ⭐ Q1. How would you handle inter-service communication in a microservices architecture using Spring Boot?

**Answer:**

- Synchronous (REST/gRPC): Use WebClient (reactive) or RestTemplate for HTTP calls
- Spring Cloud OpenFeign: Declarative REST client — interface-based HTTP calls
- Asynchronous (Messaging): RabbitMQ or Kafka for event-driven, decoupled communication
- Service Discovery: Eureka/Consul so services find each other dynamically
- Circuit Breaker: Resilience4j to prevent cascade failures

```
@FeignClient(name = 'inventory-service')
public interface InventoryClient {
    @GetMapping('/inventory/{productId}')
    int getStock(@PathVariable String productId);
}
```

---

### ⭐ Q2. Can you explain the caching mechanisms available in Spring Boot?

**Answer:**

- Spring Cache Abstraction: Annotation-based caching (@Cacheable, @CachePut, @CacheEvict)
- Caffeine: High-performance in-memory cache (recommended for local caching)
- Redis: Distributed cache for multi-instance deployments
- Ehcache: JVM-level cache with disk overflow support
- Hazelcast: Distributed in-memory data grid

---

### Q3. How would you implement caching in a Spring Boot application?

**Answer:**

1. Add starter dependency:

```
spring-boot-starter-cache + (e.g.) spring-boot-starter-data-redis
```

2. Enable caching:

```
@EnableCaching on @SpringBootApplication
```

3. Annotate methods:

```
@Cacheable(value = 'users', key = '#id')
public User findById(Long id) { return repo.findById(id).orElseThrow(); }

@CacheEvict(value = 'users', key = '#id')
public void deleteUser(Long id) { repo.deleteById(id); }
```

---

### ⭐ Q4. Your Spring Boot application is experiencing performance issues under high load.

### What steps would you take?

**Answer:**

- Profile: Use Actuator /metrics, JProfiler, or VisualVM to identify bottlenecks
- Database: Analyze slow queries, add indexes, enable connection pooling (HikariCP)
- Caching: Add Redis/Caffeine for frequent read queries
- Async: Offload heavy tasks with @Async
- Pagination: Return paginated results instead of full datasets
- Thread pool tuning: Adjust server.tomcat.threads.max
- JVM tuning: Heap size, GC algorithm (G1GC for low latency)
- Horizontal scaling: Deploy multiple instances behind a load balancer

---

### Q5. What are the best practices for versioning REST APIs in Spring Boot?

**Answer:**

- URI versioning (/api/v1/) is the most visible and widely adopted approach
- Never change the contract of an existing version — only add new versions
- Deprecate old versions with response headers (Deprecation, Sunset)
- Document all versions in Swagger/OpenAPI
- Support N-1 versions to give clients time to migrate

---

### ⭐ Q6. How does Spring Boot simplify the data access layer implementation?

**Answer:**

- Spring Data JPA: Repository interfaces (JpaRepository) with built-in CRUD and pagination
- Query methods: Derived queries from method names — findByEmailAndStatus()
- @Query: Custom JPQL or native SQL queries
- Auto-configuration: DataSource, EntityManagerFactory, and TransactionManager auto-configured
- Schema management: spring.jpa.hibernate.ddl-auto for table creation/validation

```
public interface UserRepository extends JpaRepository<User, Long> {
    Optional<User> findByEmail(String email);
    Page<User> findByStatus(String status, Pageable pageable);
}
```

---

### ⭐ Q7. What are conditional annotations and their purpose in Spring Boot?

**Answer:**

- @ConditionalOnClass: Bean registered only if specified class is on classpath
- @ConditionalOnMissingBean: Bean registered only if no bean of that type exists
- @ConditionalOnProperty: Bean registered based on property value
- @ConditionalOnExpression: SpEL-based condition
- @ConditionalOnWebApplication: Only in a web application context

```
@Bean
@ConditionalOnProperty(name = 'feature.notifications.enabled', havingValue = 'true')
public NotificationService notificationService() { return new
EmailNotificationService(); }
```

### ⭐ Q8. Explain the role of @EnableAutoConfiguration. How does Spring Boot achieve autoconfiguration internally?

**Answer:**

- @EnableAutoConfiguration triggers the AutoConfigurationImportSelector
- This reads META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
- Each listed class is conditionally evaluated using @Conditional annotations
- Only classes whose conditions are met are registered as beans
- User-defined beans take precedence via @ConditionalOnMissingBean

### Q9. What are Spring Boot Actuator endpoints?

**Answer:**

- /actuator/health — Application health (UP/DOWN)
- /actuator/info — Arbitrary application info
- /actuator/metrics — JVM memory, HTTP, custom metrics
- /actuator/env — Environment properties
- /actuator/beans — All registered Spring beans
- /actuator/loggers — View and change log levels at runtime
- /actuator/httptrace — Recent HTTP request traces
- /actuator/threaddump — Current thread dump

### ⭐ Q10. How can we secure the actuator endpoints?

**Answer:**

- Limit exposure: management.endpoints.web.exposure.include=health,info
- Change management port: management.server.port=9090 to isolate from main app
- Add Spring Security to require authentication for management endpoints

```
management.endpoints.web.exposure.include=health,info
management.endpoint.health.show-details=when-authorized
management.server.port=9090
```

### ⭐ Q11. What strategies would you use to optimize the performance of a Spring Boot application?

**Answer:**

- Connection pooling: HikariCP (default) with optimized pool size
- Caching: Redis for read-heavy data
- Async processing: @Async or messaging queues
- Database: N+1 query fixes, JPQL join fetch, lazy loading tuning
- Compression: server.compression.enabled=true
- Reduce startup time: Spring Boot lazy initialization — spring.main.lazy-initialization=true
- HTTP/2: server.http2.enabled=true

- Native image: GraalVM native compilation for instant startup

### ⭐ Q12. How can we handle multiple beans of the same type?

**Answer:**

- @Primary: Mark one bean as the default candidate for injection
- @Qualifier('beanName'): Specify which bean to inject at the injection point
- @Profile: Load different beans based on active profile

```
@Bean @Primary
public PaymentService stripePaymentService() { return new StripePaymentService(); }

@Bean
public PaymentService paypalPaymentService() { return new PayPalPaymentService(); }

@Autowired
@Qualifier('paypalPaymentService')
private PaymentService paymentService;
```

### ⭐ Q13. What are some best practices for managing transactions in Spring Boot?

**Answer:**

- Annotate service methods with @Transactional, not repositories
- Keep transactions short — avoid long-running operations inside a transaction
- Use readOnly=true for read operations for performance
- Set appropriate rollbackFor exception types
- Avoid self-invocation — proxy won't intercept internal calls
- Use propagation and isolation levels appropriately

```
@Transactional(readOnly = true)
public List<User> getAll() { return repo.findAll(); }

@Transactional(rollbackFor = Exception.class)
public void createOrder(Order order) { orderRepo.save(order); }
```

### Q14. How do you approach testing in Spring Boot applications?

**Answer:**

- Unit: JUnit + Mockito for isolated logic tests (fast, no Spring context)
- Slice tests: @WebMvcTest (web layer), @DataJpaTest (repository layer)
- Integration: @SpringBootTest for full context
- Contract tests: Spring Cloud Contract for microservice contracts
- End-to-end: TestRestTemplate / WebTestClient

### ⭐ Q15. Discuss the use of @SpringBootTest and @MockBean annotations.

**Answer:**

- @SpringBootTest: Loads complete ApplicationContext; slower but realistic
- @MockBean: Replaces a real bean in the ApplicationContext with a Mockito mock
- Use @MockBean to isolate specific layers during integration tests

```
@SpringBootTest
class UserServiceIntegrationTest {
    @MockBean UserRepository userRepository;
    @Autowired UserService userService;
    @Test
    void findUser() {
        when(userRepository.findById(1L)).thenReturn(Optional.of(new User()));
        assertNotNull(userService.findById(1L));
    }
}
```

## ⭐ Q16. What advantages does YAML offer over properties files in Spring Boot?

**Answer:**

- Hierarchical structure: Avoids repetitive prefixes (spring.datasource.url vs nested spring: datasource: url:)
- Multi-profile support: Multiple profiles in a single file using --- separator
- Better readability for complex configurations
- Native list/map support

Limitations of YAML:

- Indentation-sensitive — tabs not allowed, only spaces
- Cannot be used with @PropertySource annotation
- Parsing errors can be harder to debug

## ⭐ Q17. Explain how Spring Boot profiles work.

**Answer:**

Profiles allow you to define different configurations for different environments (dev, test, prod).

```
# application-dev.yml
spring:
  datasource:
    url: jdbc:h2:mem:testdb

# Activate:
spring.profiles.active=dev
```

- @Profile('dev'): Bean only created when dev profile is active
- Multiple profiles: spring.profiles.active=dev,messaging

## Q18. Why use Profiles in Spring Boot?

**Answer:**

- Environment-specific config: Different DB URLs, credentials, feature flags per environment
- Conditional bean loading: Enable/disable components per environment
- Testing: Use an in-memory H2 for tests, real DB for production
- Feature flags: Enable experimental features in dev but not prod

---

### ⭐ Q19. What is aspect-oriented programming in the Spring Framework?

**Answer:**

AOP allows separating cross-cutting concerns (logging, security, transactions) from business logic.

- Aspect: Class annotated with @Aspect containing cross-cutting logic
- Advice: Action taken — @Before, @After, @Around, @AfterReturning, @AfterThrowing
- Pointcut: Expression defining where advice is applied
- JoinPoint: A specific point in execution (method call)

```
@Aspect @Component
public class LoggingAspect {
    @Around('@annotation(LogExecutionTime)')
    public Object logTime(ProceedingJoinPoint pjp) throws Throwable {
        long start = System.currentTimeMillis();
        Object result = pjp.proceed();
        long duration = System.currentTimeMillis() - start;
        log.info('Execution took: {}ms', duration);
        return result;
    }
}
```

---

### ⭐ Q20. What is Spring Cloud and how is it useful for building microservices?

**Answer:**

- Spring Cloud Config: Centralized configuration management
- Eureka / Consul: Service discovery and registration
- Spring Cloud Gateway: API Gateway for routing, filtering, and rate limiting
- Resilience4j: Circuit breaker, retry, rate limiter patterns
- Spring Cloud Sleuth + Zipkin: Distributed tracing
- OpenFeign: Declarative REST client for inter-service calls
- Spring Cloud Bus: Broadcast configuration changes across services

---

### Q21. How does Spring Boot make the decision on which server to use?

**Answer:**

Spring Boot's auto-configuration checks which server starters are on the classpath using @ConditionalOnClass. The order of precedence is Tomcat > Jetty > Undertow. For reactive apps, Netty is selected when spring-boot-starter-webflux is used without spring-boot-starter-web.

---

### Q22. How to get the list of all the beans in your Spring Boot application?

**Answer:**

- Actuator endpoint: GET /actuator/beans (requires management.endpoints.web.exposure.include=beans)
- Programmatically via ApplicationContext.getBeanDefinitionNames()

---

### Q23. Describe a Spring Boot project where you significantly improved performance.

**Answer:**

Example answer for interviews: 'In a high-traffic e-commerce API, we identified N+1 query issues using Hibernate statistics. We added JPQL join fetches, introduced Redis caching for product catalog data (@Cacheable), enabled HTTP response compression, and tuned HikariCP pool size. Response times improved from 800ms to 120ms average.'

- Profile first: identify bottlenecks before optimizing
- Database fixes often yield the biggest gains
- Measure before and after every change

---

### Q24. Explain the concept of Spring Boot's embedded servlet containers.

**Answer:**

Embedded servlet containers are web servers bundled inside the application JAR. Spring Boot auto-configures them — no external server installation needed. The server lifecycle is managed by Spring Boot: starts on application startup, stops on shutdown.

- Tomcat: Default, battle-tested, supports Servlet API
- Jetty: Lightweight, lower memory footprint
- Undertow: High-performance, non-blocking IO

---

### ⭐ Q25. How does Spring Boot make DI easier compared to traditional Spring?

**Answer:**

- Auto-configuration: No XML beans.xml needed — annotations drive DI
- Component scanning: Auto-discovers @Component, @Service, @Repository in classpath
- Starters: Dependencies pre-configured for common patterns
- Property injection: @Value and @ConfigurationProperties simplify configuration injection

---

### ⭐ Q26. How does Spring Boot simplify management of application secrets?

**Answer:**

- Profiles: Different credentials per environment via application-{profile}.yml
- Environment variables: Override any property via OS environment variables
- Spring Cloud Config: Centralized config server with encryption support
- HashiCorp Vault integration: spring-cloud-vault for secrets management
- Kubernetes Secrets: Mounted as environment variables or config maps
- AWS Secrets Manager / Azure Key Vault: Cloud-native secret stores
- Never commit secrets to version control — use environment-specific property sources

### ⭐ Q27. Explain Spring Boot's approach to handling asynchronous operations.

**Answer:**

- @Async: Marks a method to run in a separate thread from a thread pool
- @EnableAsync: Required on configuration class to activate async support
- Returns CompletableFuture<T> or Future<T> for async results
- Custom executor: Configure ThreadPoolTaskExecutor for pool size tuning
- Exception handling: Implement AsyncUncaughtExceptionHandler

### Q28. How can you enable and use asynchronous methods?

**Answer:**

```
@Configuration @EnableAsync
public class AsyncConfig implements AsyncConfigurer {
    @Override
    public Executor getAsyncExecutor() {
        ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();
        executor.setCorePoolSize(5);
        executor.setMaxPoolSize(20);
        executor.setQueueCapacity(100);
        executor.initialize();
        return executor;
    }
}

@Async
public CompletableFuture<String> processAsync(String data) {
    return CompletableFuture.completedFuture(process(data));
}
```

### ⭐ Q29. How would you secure a Spring Boot application accessed by multiple users with different roles?

**Answer:**

- Spring Security: Role-based access control with @PreAuthorize or HttpSecurity
- JWT tokens: Stateless authentication carrying user roles in claims
- Method-level security: @PreAuthorize('hasRole(ADMIN)') on service methods

```
http.authorizeHttpRequests(auth -> auth
    .requestMatchers('/admin/**').hasRole('ADMIN')
    .requestMatchers('/api/**').hasAnyRole('USER', 'ADMIN')
    .anyRequest().authenticated()
);
```

### ⭐ Q30. You're creating an endpoint for file uploads. Explain how you would handle it.

**Answer:**

- Use @RequestParam MultipartFile file in the controller method
- Configure: spring.servlet.multipart.max-file-size=10MB
- Storage options: Local filesystem, AWS S3 (AWS SDK), Azure Blob Storage

```
@PostMapping('/upload')
public ResponseEntity<String> upload(@RequestParam MultipartFile file) throws
IOException {
    String filename = StringUtils.cleanPath(file.getOriginalFilename());
    Path target = Paths.get(uploadDir).resolve(filename);
    Files.copy(file.getInputStream(), target, StandardCopyOption.REPLACE_EXISTING);
    return ResponseEntity.ok('Uploaded: ' + filename);
}
```

---

### ⭐ Q31. Can you explain the difference between authentication and authorization in Spring Security?

**Answer:**

- Authentication: Verifying WHO the user is (username/password, JWT, OAuth2)
- Authorization: Verifying WHAT the authenticated user is allowed to do
- Spring Security authentication: UsernamePasswordAuthenticationFilter, JwtFilter
- Spring Security authorization: @PreAuthorize, HttpSecurity.authorizeHttpRequests()

Authentication happens first; authorization follows once identity is confirmed.

---

### Q32. After registration, your app needs to send a welcome email. Describe how.

**Answer:**

- Add spring-boot-starter-mail dependency
- Configure: spring.mail.host, spring.mail.username, spring.mail.password
- Inject JavaMailSender and use MimeMessageHelper
- Send asynchronously with @Async to avoid blocking registration response

```
@Async
public void sendWelcomeEmail(String to, String name) throws MessagingException {
    MimeMessage msg = mailSender.createMimeMessage();
    MimeMessageHelper helper = new MimeMessageHelper(msg, true);
    helper.setTo(to);
    helper.setSubject('Welcome, ' + name + '!');
    helper.setText('<h1>Welcome!</h1>', true);
    mailSender.send(msg);
}
```

---

### Q33. What is Spring Boot CLI and how to execute a Spring Boot project using Boot CLI?

**Answer:**

Spring Boot CLI lets you run Groovy scripts as Spring Boot apps:

```
spring run hello.groovy
```

Groovy example (hello.groovy):

```
@RestController
class HelloController {
    @GetMapping('/') String home() { 'Hello World!' }
}
```

### ⭐ Q34. How is Spring Security implemented in a Spring Boot application?

**Answer:**

- Add spring-boot-starter-security dependency
- Define SecurityFilterChain bean with HttpSecurity configuration
- Implement UserDetailsService to load users from DB
- Configure PasswordEncoder (BCryptPasswordEncoder recommended)

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers('/public/**').permitAll()
            .anyRequest().authenticated())
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
        .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
        .build();
}
```

### Q35. How to disable a specific auto-configuration?

**Answer:**

Same as Level I Q21 — use exclude attribute or spring.autoconfigure.exclude property. In tests, use @EnableAutoConfiguration(exclude={...}) to keep tests lean.

### ⭐ Q36. Explain the difference between cache eviction and cache expiration.

**Answer:**

- Cache Expiration (TTL): Entry is automatically removed after a time-to-live period
- Cache Eviction: Entry is removed due to a policy — LRU (Least Recently Used), LFU, explicit @CacheEvict
- Eviction is demand-driven (size limits or explicit removal)
- Expiration is time-driven (stale data cleanup)

### ⭐ Q37. If you had to scale a Spring Boot application to handle high traffic, what strategies would you use?

**Answer:**

- Horizontal scaling: Deploy multiple instances behind a load balancer (Nginx, AWS ALB)

- Stateless design: Store session in Redis, not in-memory
- Database: Read replicas, connection pool tuning, query optimization
- Caching: Redis for hot data, CDN for static assets
- Async processing: Offload work to Kafka/RabbitMQ queues
- Auto-scaling: Kubernetes HPA or AWS Auto Scaling Groups
- Rate limiting: Protect against abuse using Spring Cloud Gateway or Bucket4j

---

### ⭐ Q38. Describe how to implement security in a microservices architecture using Spring Boot and Spring Security.

**Answer:**

- API Gateway: Single authentication entry point (Spring Cloud Gateway + Spring Security OAuth2)
- JWT: Services validate tokens without calling auth service for every request
- OAuth2 + OIDC: Industry standard for authentication (Keycloak, Auth0, Okta)
- mTLS: Mutual TLS for service-to-service authentication
- @PreAuthorize at service level for fine-grained authorization

---

### Q39. How is session management configured in Spring Boot for distributed systems?

**Answer:**

- Stateless (preferred for microservices): Use JWT — no server-side session
- Spring Session + Redis: Share session across multiple instances
- Configure: spring-session-data-redis + @EnableRedisHttpSession
- SessionCreationPolicy.STATELESS: No HttpSession created by Spring Security

---

### ⭐ Q40. How would you handle API rate limits and failures from external APIs?

**Answer:**

- Resilience4j CircuitBreaker: Open circuit after threshold failures
- Retry: Automatic retry with exponential backoff
- Rate Limiter: Limit calls to external API per time window
- Bulkhead: Isolate external API calls in separate thread pools
- Fallback: Return cached data or default response when API is unavailable

```
@CircuitBreaker(name = 'externalApi', fallbackMethod = 'fallback')
@Retry(name = 'externalApi')
public String callExternalApi() { return client.get(); }

public String fallback(Exception e) { return cachedData.get(); }
```

---

### ⭐ Q41. How would you manage externalized configuration in a microservices architecture?

**Answer:**

- Spring Cloud Config Server: Central config server backed by Git repository
- Kubernetes ConfigMaps and Secrets: Native K8s config management

- HashiCorp Vault: Dynamic secrets management
- Environment variables: 12-factor app approach
- @ConfigurationProperties: Type-safe binding of configuration to POJOs

```
@ConfigurationProperties(prefix = 'app')
@Data
public class AppProperties {
    private String apiKey;
    private int timeoutSeconds;
}
```

### Q42. Can we create a non-web application in Spring Boot?

**Answer:**

Yes — same as Level I Q22. Use spring.main.web-application-type=none. Implement CommandLineRunner or ApplicationRunner for application logic.

### Q43. What does the @SpringBootApplication annotation do internally?

**Answer:**

Same as Level I Q9 — it combines @SpringBootConfiguration + @EnableAutoConfiguration + @ComponentScan.

### Q44. How does Spring Boot support internationalization (i18n)?

**Answer:**

- MessageSource: Bean that resolves messages from properties files
- Create messages_en.properties, messages_fr.properties, etc.
- LocaleResolver: Determines locale from request (Accept-Language header or session)

```
@Bean
public MessageSource messageSource() {
    ReloadableResourceBundleMessageSource source = new
ReloadableResourceBundleMessageSource();
    source.setBasename('classpath:messages');
    source.setDefaultEncoding('UTF-8');
    return source;
}
```

### Q45. What is Spring Boot DevTools used for?

**Answer:**

Same as Level I Q35. Additionally, DevTools supports remote debugging and remote application restart for servers — configure spring.devtools.remote.secret for remote restart.

### ⭐ Q46. How can you mock external services in a Spring Boot test?

**Answer:**

- @MockBean: Replaces a Spring bean with a Mockito mock in the context
- WireMock: Starts a stub HTTP server to simulate external REST APIs
- MockRestServiceServer: Mock RestTemplate calls

```
@SpringBootTest
class PaymentServiceTest {
    @Autowired private WireMockServer wireMock;
    @Test
    void processPayment() {
        wireMock.stubFor(post('/payment').willReturn(ok().withBody('{status:ok}')));
        assertEquals('ok', paymentService.charge(order));
    }
}
```

---

### Q47. How do you mock microservices during testing?

**Answer:**

- WireMock for HTTP service stubs
- TestContainers for real infrastructure (Kafka, Redis, databases) in containers
- Spring Cloud Contract for consumer-driven contract testing
- @MockBean for in-process Spring bean mocking

---

### ⭐ Q48. Explain the process of creating a Docker image for a Spring Boot application.

**Answer:**

- Option 1: Dockerfile

```
FROM eclipse-temurin:21-jre
COPY target/app.jar app.jar
ENTRYPOINT ['java', '-jar', 'app.jar']
```

- Option 2: Spring Boot Maven plugin (Buildpack — no Dockerfile needed)

```
./mvnw spring-boot:build-image -Dspring-boot.build-image.imageName=myapp:1.0
```

- Option 3: Layered JARs — optimize Docker layer caching by separating dependencies from app code

---

### ⭐ Q49. Discuss the configuration of Spring Security to address common security concerns.

**Answer:**

- CSRF: Disable for stateless REST APIs; enable for stateful web apps
- CORS: Configure CorsConfigurationSource for cross-origin requests
- Password encoding: Always use BCryptPasswordEncoder
- HTTPS: Enforce HTTPS with HTTP Strict Transport Security (HSTS)
- Input validation: @Valid + Bean Validation to prevent injection attacks
- Security headers: X-Frame-Options, X-Content-Type-Options, CSP

---

### ⭐ Q50. Discuss how you would secure a Spring Boot application using JWT.

**Answer:**

- Step 1: User logs in with credentials → returns signed JWT
- Step 2: Client sends JWT in Authorization: Bearer <token> header
- Step 3: JwtAuthenticationFilter validates token on every request
- Step 4: Extract claims, set SecurityContext
- Use JJWT or Nimbus JOSE+JWT library
- Sign with RSA (asymmetric) for microservices so services can verify without the secret key
- Short expiry (15 min) + refresh tokens for security

---

### ⭐ Q51. How can Spring Boot applications be made more resilient, especially in microservices?

**Answer:**

- Resilience4j: Circuit breaker, retry, rate limiter, bulkhead, time limiter
- Timeout configuration: Prevent hanging calls
- Fallback methods: Graceful degradation
- Health checks: Actuator + K8s liveness/readiness probes
- Retry with backoff: Exponential backoff to avoid thundering herd
- Dead letter queues: Handle failed message processing

---

### Q52. Explain the conversion of business logic into serverless functions with Spring Cloud Function.

**Answer:**

- Spring Cloud Function wraps business logic as java.util.function.Function<Input, Output>
- Deploy to AWS Lambda, Azure Functions, or GCP Cloud Functions
- Same code runs locally, as serverless, or as a standard Spring Boot app

```
@Bean
public Function<String, String> uppercase() {
    return String::toUpperCase;
}
```

---

### ⭐ Q53. How can Spring Cloud Gateway be configured for routing, security, and monitoring?

**Answer:**

- Routing: Define routes with predicates (Path, Host, Method)
- Filters: Add/remove headers, rate limiting, circuit breaking, authentication
- Security: Integrate OAuth2 resource server or custom JWT filter
- Monitoring: Expose Actuator metrics + Zipkin for distributed tracing

```
spring:
  cloud:
    gateway:
      routes:
```

```
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/api/users/**
          filters:
            - name: CircuitBreaker
              args: { name: userCB, fallbackUri: forward:/fallback }
```

### Q54. How would you manage and monitor asynchronous tasks in Spring Boot?

**Answer:**

- @Async with CompletableFuture: Track future completion status
- ThreadPoolTaskExecutor metrics: Expose via Actuator
- Database task tracking: Persist task status (PENDING, RUNNING, DONE, FAILED)
- Spring Batch: For long-running batch jobs with step-level monitoring
- Message queue DLQ: Failed tasks end up in dead-letter queue for retry

### ⭐ Q55. Your application needs to process notifications asynchronously using a message queue.

**Answer:**

- Add spring-boot-starter-amqp (RabbitMQ) or spring-kafka dependency
- Configure connection: spring.rabbitmq.host, spring.rabbitmq.port
- Send message: Inject RabbitTemplate or KafkaTemplate and call convertAndSend()
- Receive: Annotate method with @RabbitListener(queues = 'notifications')

```
@Service
public class NotificationPublisher {
    @Autowired RabbitTemplate rabbitTemplate;
    public void notify(String message) {
        rabbitTemplate.convertAndSend('notifications', message);
    }
}
```

### Q56. Configure Spring Security for basic form-based authentication.

**Answer:**

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    return http
        .authorizeHttpRequests(a -> a.requestMatchers('/public/**').permitAll()
                                     .anyRequest().authenticated())
        .formLogin(form -> form.loginPage('/login').defaultSuccessUrl('/home'))
        .logout(logout -> logout.logoutSuccessUrl('/login?logout'))
        .build();
}
```

### Q57. How to tell an auto-configuration to back away when a bean exists?

**Answer:**

Use @ConditionalOnMissingBean on the auto-configuration @Bean method:

```
@Bean
@ConditionalOnMissingBean
public DataSource dataSource() {
    return DataSourceBuilder.create().build();
}
```

If a user defines their own DataSource bean, Spring Boot's auto-configured DataSource will not be created.

---

### ⭐ Q58. How to deploy Spring Boot web applications as JAR and WAR files?

**Answer:**

- JAR (default): packaging=jar; run with java -jar; includes embedded server
- WAR: Change packaging to war; extend SpringBootServletInitializer

```
// For WAR deployment:
@SpringBootApplication
public class MyApp extends SpringBootServletInitializer {
    @Override
    protected SpringApplicationBuilder configure(SpringApplicationBuilder b) {
        return b.sources(MyApp.class);
    }
}
```

---

### Q59. What does it mean that Spring Boot supports relaxed binding?

**Answer:**

Relaxed binding allows property names to be specified in multiple formats and Spring Boot will bind them to the same property:

- spring.jpa.show-sql (kebab-case)
- spring.jpa.showSql (camelCase)
- SPRING_JPA_SHOW_SQL (environment variable uppercase)
- spring.jpa.show_sql (underscore)

All four bind to the same showSql property.

---

### ⭐ Q60. Discuss the integration of Spring Boot applications with CI/CD pipelines.

**Answer:**

- Build: Maven/Gradle triggered by git push (GitHub Actions, Jenkins, GitLab CI)
- Test: Unit tests + integration tests run automatically on every commit
- Code quality: SonarQube analysis for code coverage and security issues
- Docker build: spring-boot:build-image creates OCI image
- Push to registry: Docker Hub, ECR, GCR

- Deploy: Kubernetes rolling updates or Helm chart deployments
- Canary/Blue-Green: Zero-downtime deployments

### Q61. Can we override or replace the Embedded Tomcat server in Spring Boot?

**Answer:**

Yes — exclude the Tomcat starter and add Jetty or Undertow (same as Level I Q57). You can also programmatically customize Tomcat via WebServerFactoryCustomizer<TomcatServletWebServerFactory>.

### Q62. How to resolve Whitelabel Error Page in Spring Boot?

**Answer:**

- Option 1: Create a custom error controller implementing ErrorController
- Option 2: Create error/404.html, error/500.html templates under resources/templates/
- Option 3: Disable it: server.error.whitelabel.enabled=false and handle manually

```
@Controller
public class CustomErrorController implements ErrorController {
    @GetMapping('/error')
    public String error(HttpServletRequest req, Model model) {
        model.addAttribute('code', req.getAttribute(ERROR_STATUS_CODE));
        return 'error';
    }
}
```

### ⭐ Q63. How can you implement pagination in a Spring Boot application?

**Answer:**

- Spring Data supports Pageable interface and Page<T> return type

```
Page<User> findByStatus(String status, Pageable pageable);

// Controller:
@GetMapping
public Page<User> getUsers(
    @RequestParam(defaultValue='0') int page,
    @RequestParam(defaultValue='10') int size) {
    return service.findAll(PageRequest.of(page, size, Sort.by('name')));
}
```

Response includes content, totalElements, totalPages, number, size.

### Q64. How to handle a 404 error in Spring Boot?

**Answer:**

- @ControllerAdvice with @ExceptionHandler(NoHandlerFoundException.class)
- Custom ErrorController for all error codes

- In properties: spring.web.resources.add-mappings=false and spring.mvc.throw-exception-if-no-handler-found=true

### ⭐ Q65. How can Spring Boot be used to implement event-driven architectures?

**Answer:**

- Spring Application Events: ApplicationEvent + @EventListener for in-process events
- Spring Kafka: Producer/consumer for distributed event streaming
- Spring AMQP: RabbitMQ for message-based events
- @TransactionalEventListener: Publish event after transaction commits

```
@Component
public class UserRegisteredHandler {
    @EventListener
    @Async
    public void handle(UserRegisteredEvent event) {
        emailService.sendWelcome(event.getEmail());
    }
}
```

### Q66. What are the basic Annotations that Spring Boot offers?

**Answer:**

- @SpringBootApplication, @RestController, @Controller, @Service
- @Repository, @Component, @Configuration, @Bean
- @Autowired, @Qualifier, @Primary, @Value
- @GetMapping, @PostMapping, @PutMapping, @DeleteMapping, @RequestMapping
- @RequestBody, @ResponseBody, @PathVariable, @RequestParam
- @Transactional, @Cacheable, @CacheEvict, @Async
- @Profile, @Conditional, @ConditionalOnProperty, @ConditionalOnMissingBean
- @SpringBootTest, @MockBean, @WebMvcTest, @DataJpaTest

### ⭐ Q67. Discuss the integration and use of distributed tracing in Spring Boot applications.

**Answer:**

- Micrometer Tracing (Spring Boot 3+): Replaces Spring Cloud Sleuth
- Zipkin: Distributed tracing visualization — add spring-zipkin dependency
- OpenTelemetry: Vendor-neutral tracing standard
- Each request gets a trace ID propagated through all services via HTTP headers
- Correlate logs using MDC with traceId for end-to-end request visibility

```
management.tracing.sampling.probability=1.0  # 100% sampling in dev
spring.zipkin.base-url=http://zipkin:9411
```

### Q68. Your application needs to store and retrieve files from a cloud storage service.

**Answer:**

- AWS S3: Use AWS SDK (software.amazon.awssdk:s3) — configure region + credentials
- Azure Blob Storage: azure-spring-boot-starter-storage
- GCP Cloud Storage: spring-cloud-gcp-starter-storage

```
// AWS S3 upload example
s3Client.putObject(PutObjectRequest.builder()
    .bucket(bucketName).key(filename).build(),
    RequestBody.fromInputStream(file.getInputStream(), file.getSize()));
```

---

### ⭐ Q69. You decide to implement rate limiting on your API endpoints. Describe a simple approach.

**Answer:**

- Bucket4j: Token bucket algorithm — in-memory or Redis-backed
- Spring Cloud Gateway: Built-in RequestRateLimiter filter with Redis
- Custom filter: Implement OncePerRequestFilter with IP-based counters

```
// Spring Cloud Gateway config:
filters:
  - name: RequestRateLimiter
    args:
      redis-rate-limiter.replenishRate: 10
      redis-rate-limiter.burstCapacity: 20
      redis-rate-limiter.requestedTokens: 1
```

---

### Q70. Your application requires a 'soft delete' feature. How would you implement it?

**Answer:**

- Add a boolean deletedAt timestamp or deleted flag to the entity
- Use @SQLRestriction (Hibernate 6+) or @Where (older) to filter deleted records
- Override delete method to update flag instead of removing row

```
@Entity
@SQLRestriction('deleted = false')
public class Product {
    private boolean deleted = false;
    private LocalDateTime deletedAt;
}

// Service:
public void delete(Long id) {
    Product p = repo.findById(id).orElseThrow();
    p.setDeleted(true);
    p.setDeletedAt(LocalDateTime.now());
    repo.save(p);
}
```

---

### ⭐ Q71. Describe how you would use Spring WebFlux to build a non-blocking reactive REST API.

**Answer:**

- Use spring-boot-starter-webflux (replaces spring-boot-starter-web)
- Return Mono<T> (single item) or Flux<T> (stream) from controllers
- Use R2DBC for non-blocking reactive database access
- Default server: Netty (non-blocking event loop)
- Handles thousands of concurrent connections with a small thread pool

```
@RestController
public class UserController {
    @GetMapping('/users')
    public Flux<User> all() { return userRepo.findAll(); }

    @GetMapping('/users/{id}')
    public Mono<User> one(@PathVariable Long id) {
        return userRepo.findById(id).switchIfEmpty(Mono.error(new
NotFoundException(id)));
    }
}
```

# Spring Boot Quick Reference Cheat Sheet

*A consolidated reference of the most important Spring Boot concepts, annotations, and configurations.*

## Core Annotations

| Annotation | Purpose |
|---|---|
| `@SpringBootApplication` | Entry point: combines @Configuration + @ComponentScan + @EnableAutoConfiguration |
| `@RestController` | REST API controller — @Controller + @ResponseBody combined |
| `@Service` | Business logic layer bean |
| `@Repository` | Data access layer — adds exception translation |
| `@Component` | Generic Spring-managed bean |
| `@Autowired` | Dependency injection (prefer constructor injection) |
| `@Bean` | Factory method — defines a Spring bean in a @Configuration class |
| `@Value` | Inject a property value: @Value('${property.name}') |
| `@ConfigurationProperties` | Type-safe property binding to a POJO |
| `@Transactional` | Wrap method/class in a database transaction |
| `@Async` | Run method asynchronously in a thread pool |
| `@Cacheable` | Cache method result using the cache abstraction |
| `@CacheEvict` | Remove entry from cache on method invocation |
| `@Profile` | Register bean only when specified profile is active |
| `@ConditionalOnMissingBean` | Register bean only if no bean of that type exists |
| `@ConditionalOnProperty` | Register bean based on a property value |
| `@SpringBootTest` | Integration test: loads full ApplicationContext |
| `@MockBean` | Replace a Spring bean with a Mockito mock in tests |
| `@WebMvcTest` | Test only the web layer (controllers) without full context |
| `@DataJpaTest` | Test only JPA layer with in-memory H2 database |
| `@ExceptionHandler` | Handle specific exception in a controller or @ControllerAdvice |
| `@RestControllerAdvice` | Global exception handler for all REST controllers |
| `@PreAuthorize` | Method-level security expression: @PreAuthorize('hasRole(ADMIN)') |
| `@EnableCaching` | Activate Spring's cache abstraction |
| `@EnableAsync` | Activate asynchronous method execution |

## Key Properties (application.properties / application.yml)

| Property | Description |
|---|---|
| `server.port=8080` | Change embedded server port |
| `spring.profiles.active=dev` | Set active profile |
| `spring.datasource.url=...` | Database connection URL |
| `spring.jpa.hibernate.ddl-auto=update` | Schema generation strategy |
| `spring.jpa.show-sql=true` | Log SQL queries |
| `logging.level.root=DEBUG` | Set root log level |
| `spring.main.lazy-initialization=true` | Improve startup time |
| `management.endpoints.web.exposure.include=*` | Expose all Actuator endpoints |
| `spring.autoconfigure.exclude=...` | Disable specific auto-configuration |
| `spring.cache.type=redis` | Configure cache provider |
| `spring.servlet.multipart.max-file-size=10MB` | Max file upload size |
| `server.compression.enabled=true` | Enable HTTP response compression |
| `spring.main.web-application-type=none` | Create non-web Spring Boot app |

## HTTP Status Codes for REST APIs

| Code | Meaning | When to Use |
|---|---|---|
| 200 OK | Success | GET, PUT, PATCH returning data |
| 201 Created | Resource created | POST that creates a resource |
| 204 No Content | Success, no body | DELETE, PUT with no response body |
| 400 Bad Request | Invalid input | Validation failures |
| 401 Unauthorized | Not authenticated | Missing or invalid credentials |
| 403 Forbidden | Not authorized | Authenticated but no permission |
| 404 Not Found | Resource absent | Resource with given ID doesn't exist |
| 409 Conflict | State conflict | Duplicate resource or version conflict |
| 422 Unprocessable | Semantic error | Business rule violation |
| 500 Internal Error | Server error | Unexpected server-side exception |

## Spring Boot Starter Dependencies Quick Reference

| Starter | Includes |
|---|---|
| spring-boot-starter-web | Tomcat + Spring MVC + Jackson (REST APIs) |
| spring-boot-starter-data-jpa | Hibernate + Spring Data JPA |
| spring-boot-starter-security | Spring Security |
| spring-boot-starter-test | JUnit 5 + Mockito + AssertJ + MockMvc |
| spring-boot-starter-actuator | Health, metrics, monitoring endpoints |
| spring-boot-starter-cache | Spring Cache abstraction |
| spring-boot-starter-amqp | RabbitMQ integration |
| spring-boot-starter-mail | JavaMailSender for email |
| spring-boot-starter-validation | Bean Validation (Hibernate Validator) |
| spring-boot-starter-webflux | Reactive web with Netty + WebFlux |
| spring-boot-starter-data-redis | Spring Data Redis (Jedis/Lettuce) |

## Microservices Architecture Quick Checklist

- Service discovery: Eureka or Consul for dynamic routing
- API Gateway: Spring Cloud Gateway — single entry point
- Config management: Spring Cloud Config Server + Git
- Communication: Feign (sync) + Kafka/RabbitMQ (async)
- Resilience: Resilience4j — circuit breaker, retry, bulkhead
- Security: OAuth2 + JWT — gateway handles auth; services verify token
- Observability: Micrometer + Zipkin + Prometheus + Grafana
- Containerization: Docker + Kubernetes for orchestration

## Frequently Asked Questions Summary (⭐)

**These questions appear most often in Spring Boot technical interviews:**

- What is Spring Boot and how does it differ from Spring?

- What does @SpringBootApplication do internally?
- Explain auto-configuration and how it works
- How to handle exceptions globally in Spring Boot?
- What is Spring Boot Actuator and how to secure it?
- How do you implement caching in Spring Boot?
- Explain Spring Boot profiles and how they work
- Difference between @Mock and @InjectMocks in Mockito
- How does Spring Security authentication vs authorization work?
- How do you implement JWT security in Spring Boot?
- How to handle inter-service communication in microservices?
- How do you scale a Spring Boot application for high traffic?
- How do you implement pagination with Spring Data JPA?
- How to create a Docker image for a Spring Boot application?

## Good luck with your Spring Boot interviews! ☕

*Practice coding, understand internals, and explain your thinking clearly.*